

Named Entity Recognition (NER) Project Report- 13/10/25(Monday):

1. Objective:

The goal of this project is to extract named entities (like people, organizations, locations, dates, money, etc.) from raw text using Python's Natural Language Toolkit (NLTK). This project processes text from scratch, without relying on pre-labeled datasets.

2. Tools & Libraries:

- Python 3
 - NLTK (Natural Language Toolkit)
 - Google Colab (development & execution environment)
-

3. Methodology / Approach:

Step 1: Input Text

- Text is provided as a list of sentences:

texts = [

"Apple is looking at buying U.K. startup for \$1 billion.",

"Elon Musk founded SpaceX in 2002 in California."

]

Step 2: Tokenization

- The text is split into tokens (words) using `word_tokenize()`.

Mathematical Representation:

For a sentence SSS with words $w_1, w_2, \dots, w_n$

$\text{Tokens} = \{w_1, w_2, \dots, w_n\}$

Step 3: Part-of-Speech (POS) Tagging

- Each token is assigned a POS tag (e.g., NNP = proper noun, VBZ = verb).
- Uses averaged perceptron tagging model.

Notation:

$\text{POS_Tags} = \{(w_i, t_i) | i = 1..n\}, t_i \in \text{POS tag set}$

Step 4: Named Entity Recognition (NER)

- The `ne_chunk()` function creates a parse tree that groups tokens into named entities.
- Common NER classes:
 - PERSON → Names of people
 - ORGANIZATION → Companies, institutions
 - GPE → Countries, cities, states

Logic:

NER can be considered as a sequence labeling problem, where each token w_i is assigned a label y_i from a set of named entity classes:

$y_i \in \{\text{PERSON, ORGANIZATION, GPE, ...}\}$

- Tree-based chunking uses a decision-tree classifier under the hood.
-

Step 5: Manual Correction

- Common misclassifications (e.g., “Apple”) are corrected automatically:

if `entity_name` in `companies`:

`entity_type = "ORGANIZATION"`

Step 6: Output

- Entities are extracted as tuples (`entity`, `entity_type`):

`[('Apple', 'ORGANIZATION'), ('Elon Musk', 'PERSON'), ('SpaceX', 'ORGANIZATION')]`

4. Advantages of This Approach:

1. From Scratch — works with your own text, no pre-labeled dataset required.
 2. Extensible — can process multiple sentences or entire documents.
 3. Accurate — corrects common entity misclassifications.
 4. Colab-Friendly — compatible with cloud notebooks.
-

5. Limitations:

1. NLTK NER is rule-based & chunker-based — may misclassify some entities.
2. Less accurate than modern deep learning NER models (e.g., spaCy, BERT-based NER).

6. Conclusion:

This project demonstrates how to extract named entities from raw text using NLTK. The project is efficient, works on multiple sentences, and can be extended to process real datasets. With minor enhancements (company correction, text tree visualization